

CivicMitra — Comprehensive API Documentation & Analysis

Version: v1.0

Generated for: CivicMitra Project (SIH) — Full API Design, Security & Operational Analysis

Table of Contents

1. Introduction & API Principles
2. Authentication & Security Patterns
3. Error Handling & Common Schemas
4. Rate Limiting, Pagination & Idempotency
5. File Uploads, Media Storage & Signed URLs
6. Geospatial / Routing Data Formats
7. API Groups (detailed endpoints + 10-point analysis each):
 - Auth APIs
 - Citizen Report APIs
 - Admin / Management APIs
 - Field Officer APIs
 - Notifications & Communication APIs
 - Analytics & Export APIs
 - System & Health APIs
 - ML & Future Integration APIs
8. Webhooks, Integrations & 3rd-party considerations
9. Monitoring, Logging & Observability
10. Deployment, Versioning & Backwards Compatibility
11. Appendix: Example Request / Response Samples
12. Checklist: Pre-launch API Validation

1. Introduction & API Principles

This document is a comprehensive API reference and operational analysis for CivicMitra — a crowdsourced civic issue reporting and resolution platform. The APIs are designed to support a mobile-first PWA for citizens and a role-based administrative portal for government staff. Services must be secure, scalable, and compliant with data residency norms. High-level API principles: - RESTful design with resource-oriented endpoints (versioned under /api/v1). - JSON as the default interchange format. Multipart/form-data for file uploads. - Strong authentication (JWT/OIDC) and Role-Based Access Control (RBAC). - Idempotent endpoints for create/assign operations where applicable. - Clear error schemas and consistent status codes. - Support for pagination, filtering, and sorting on list endpoints.

API Host & Base Path

Base URL (example): `https://api.civicmitra.jh.gov.in/api/v1`

All endpoints in this document are prefixed with /api/v1 unless otherwise noted.

2. Authentication & Security Patterns

Authentication: - Use OAuth2 / OpenID Connect (Keycloak recommended) for Admin users. - Use JWT bearer tokens for API access. Token lifetimes: access token short (e.g., 15m); refresh tokens longer (e.g., 30d). - For citizens: support OTP-based phone verification and optional email signup (Firebase Auth or similar). Roles & RBAC: - Roles: AnonymousReporter (public), RegisteredUser (citizen), FieldOfficer, DeptAdmin, SuperAdmin, Analyst. - Each endpoint documents required role(s). Transport & Storage Security: - TLS 1.3 enforced. HSTS header present. - Encryption at rest: AES-256 using KMS-managed keys. - Signed URLs for temporary media access (expiry e.g., 1 hour). Other protections: - Rate limiting per IP and per user for write endpoints. - WAF in front of API gateway. DDoS protection enabled.

Auth Endpoints (Summary)

POST /auth/login, POST /auth/register, POST /auth/refresh, POST /auth/logout, GET /auth/profile

3. Error Handling & Common Schemas

Standard error response (JSON): { "error": { "code": "INVALID_REQUEST", "message": "Human-readable explanation", "details": { "field": "reason", ... }, "trace_id": "uuid" } } HTTP status codes: - 200 OK (successful GET) - 201 Created (resource created) - 202 Accepted (accepted for processing, async) - 204 No Content (successful deletion) - 400 Bad Request (validation errors) - 401 Unauthorized (missing/invalid token) - 403 Forbidden (insufficient role) - 404 Not Found - 409 Conflict (idempotency/resource conflict) - 422 Unprocessable Entity (domain validation) - 429 Too Many Requests (rate limit) - 500 Internal Server Error All error responses must include a trace_id for correlation with server logs.

Common Schemas

Report Object (summary):

```
{ "id": "CM-20250920-ABC123", "tracking_id": "CM-ABC123", "category": "pothole", "title": "Short title", "description": "...", "photos": ["https://.../img1.jpg"], "voice_notes": ["https://.../note1.mp3"], "location": { "lat": 23.34, "lng": 85.30, "address": "Near XYZ", "ward_id": "WARD-12"}, "priority": "medium", "status": "new", "reporter": { "id": null, "phone": "+91...", "anonymous": true }, "assigned_to": null, "department": "public_works", "created_at": "ISO8601", "updated_at": "ISO8601", "history": [{"status": "new", "by": "system", "at": "..."}], "sla_due_at": "ISO8601" }
```

4. Rate Limiting, Pagination & Idempotency

Rate Limiting: - Apply global rate limits (e.g., 100 requests/min per IP) and per-user limits (e.g., 30 create reports/hour). - Return 429 with Retry-After header on limit hit. - Apply stricter limits for unauthenticated endpoints.

Pagination: - Use cursor-based pagination or offset-based with page & size parameters. - Standard: GET /admin/reports?page=1&size=25&sort=-created_at

Idempotency: - For write endpoints (POST /reports, PUT /admin/reports/{id}/assign), support an Idempotency-Key header. - If server receives same Idempotency-Key, it should return same result without duplicating resource.

Caching: - Use Cache-Control for GET responses where appropriate. - Do not cache sensitive user-specific endpoints.

5. File Uploads, Media Storage & Signed URLs

Uploads: - Use multipart/form-data for uploads with fields for JSON payload and files. - Client-side image compression recommended (max 2MB per image). - Acceptable file types: image/jpeg, image/png, audio/mpeg, audio/ogg. - Server validates MIME type and scans for malware. Storage: - Store media in object storage (S3/GCS) with path per report: /reports/{id}/images/{filename} - Generate temporary signed GET URLs for clients to view images. Direct Upload Pattern (recommended): 1. Client requests upload URL: POST /uploads/initiate {filename, contentType, reportId} 2. Server returns signed PUT URL (expires short). 3. Client PUTs object to signed URL. 4. Client notifies server with POST /uploads/complete {reportId, fileKey, size, checksum}

Advantages: reduces load on API servers, improves throughput.

6. Geospatial / Routing Data Formats

Location data: - Use WGS84 coordinates: { "lat": 23.3456, "lng": 85.3092 } - Accept optional address and ward_id fields. - Backend uses PostGIS to resolve lat/lng to ward/panchayat polygons for routing. Routing decision: - Routing engine input: {category, lat, lng, urgencyFlag} - Output: {department_id, ward_id, suggested_officer_id, sla_days} Geo queries: - Use ST_Contains() for point-in-polygon ward lookup. - For duplicate detection: compute proximity radius (e.g., 50m) + time window (e.g., 7 days).

Auth API - Register

Endpoint: POST /auth/register

Purpose: Register new user (citizen or staff). For citizens support OTP phone verification flow; for staff require admin approval.

Auth / Roles Required: Public (for citizens) / SuperAdmin (for staff)

Request Schema / Params:

```
{ "type": "citizen" | "staff", "name": "string", "phone": "+91xxxxxxxxxx", "email": "optional", "password": "if staff", "ward_id": "optional" }
```

Response Schema / Example:

```
201 Created { "user_id": "USER-123", "status": "pending_verification", "message": "OTP sent" }
```

Key Error Cases:

400: invalid phone; 409: user exists; 422: weak password

Rate Limiting:

Unauthenticated create: 30 req/hour per IP

Idempotency:

Clients must use Idempotency-Key to ensure single registration attempt when uncertain

Validation Rules:

Phone must be validated via OTP; password rules for staff (min 12 chars + complexity)

Security Considerations:

Store minimal PII; rate-limit to prevent enumeration; verify phone via OTP; staff creation via admin UI

SLA / Processing Considerations:

Citizen OTP verification expected within 5 minutes; staff account provisioning may require manual approval

Audit Logging:

Log registration event with actor IP, user agent; do not store OTP in logs

Observability / Metrics:

Metric: registrations/day, otp_success_rate, suspended_accounts

Example Request / Response:

```
Request: POST /auth/register { "type": "citizen", "name": "Anita", "phone": "+919876543210" }  
Response: 201 Created { "user_id": "USER-987", "status": "otp_sent" }
```

Auth API - Login / OTP Verify

Endpoint: POST /auth/login

Purpose: Authenticate user. Citizens use OTP flow; staff can use password + 2FA.

Auth / Roles Required: Public

Request Schema / Params:

```
{ "phone": "+91...", "otp": "123456" // or password for staff }
```

Response Schema / Example:

```
200 OK { "access_token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5bnVpdC9hdXN0cm9kaW8iLCJ1d2kiOiJkZWZlcnR1eSIsImF1dG8iOiJ1b250b29udGVybm9uLmNpdGUiLCJleHAiOjE5ODQwMDEyMDAsImF0IjoiYWRtaWUiLCJpc2giOiJ1b250b29udGVybm9uLmNpdGUiLCJyb2wiOiJjaXRizen0iLCJ0eXAiOiJKV1QiLCJrZXkiOiJ1b250b29udGVybm9uLmNpdGUtYWRtaWUiLCJqdGkiOiJ1b250b29udGVybm9uLmNpdGUtYWRtaWUiLCJiaWF0eSI6MTUxOTg0NDQ0fQ==", "refresh_token":"...", "expires_in":900, "role":"citizen" }
```

Key Error Cases:

400: missing fields; 401: invalid OTP; 429: too many attempts

Rate Limiting:

Login attempts: 10 attempts per hour per phone

Idempotency:

Idempotency not required

Validation Rules:

OTP length 4-6 digits; passwords hashed with bcrypt/argon2

Security Considerations:

Ensure brute-force protection; CAPTCHA on repeated failures; staff 2FA required

SLA / Processing Considerations:

Access token short-lived; refresh tokens rotated

Audit Logging:

Log login attempts with result and IP; alert on suspicious activity

Observability / Metrics:

Metric: logins/day, failed login rate, average auth latency

Example Request / Response:

```
Request: POST /auth/login { "phone":"+919876543210", "otp":"123456" } Response: 200 OK { "access_token":"..", "refresh_token":"..", "expires_in":900 }
```

8. Webhooks, Integrations & 3rd-party considerations

Provide a webhooks subsystem for downstream integrations: - Example events: report.created, report.assigned, report.resolved, sla.breached - Endpoint registration: POST /integrations/webhooks {url, events, secret, active} - Deliver with HMAC-SHA256 signature header; retry with exponential backoff on 5xx. - Keep delivery logs and dead-letter queue. Third-party providers: - SMS gateway (Twilio/MSG91), Email (SES/SendGrid), Maps (Mapbox/Google), Auth (Keycloak/Firebase). - Encapsulate provider clients behind adapters to allow swapping.

9. Monitoring, Logging & Observability

- Metrics: Prometheus for business metrics and infra metrics. Expose /metrics endpoint. - Tracing: OpenTelemetry for distributed tracing. Include trace_id in logs & error responses. - Logs: Structured JSON logs, ship to ELK / Cloud Logging. - Alerts: SLA breach alerts, high error-rate, unusual traffic patterns. - Dashboards: Grafana for ops; Metabase for product/analyst dashboards.

10. Deployment, Versioning & Backwards Compatibility

- Version APIs under /api/v1. When incompatible changes required, create /api/v2. - Use feature flags for gradual rollout (LaunchDarkly or homebrew). - CI/CD: GitHub Actions -> build/test -> deploy to staging -> smoke tests -> deploy to production. - Blue/green or canary deployments recommended for zero-downtime. - Database migrations: use Liquibase or Flyway with careful rollback plans.

11. Appendix: Example Requests & Error Examples

Sample: Create Report (signed URL pattern) 1) Client POST /uploads/initiate -> server returns signed PUT url. 2) Client PUT to signed URL with media. 3) Client POST /reports with payload referencing fileKey(s). 4) Server returns 201 Created. Sample Error: 400 Bad Request { "error": {"code": "INVALID_LOCATION", "message": "Missing or invalid lat/lng", "trace_id": "..."} }

12. Checklist: Pre-launch API Validation

- Define and document all endpoints with request/response examples.
- Implement authentication & RBAC and test role mappings.
- Implement rate limiting and idempotency for create endpoints.
- Enable signed URL uploads and validate file types/sizes.
- Implement PostGIS ward lookup and test auto-routing accuracy.
- Setup logging & tracing (OpenTelemetry) and dashboards.
- Setup automated tests: unit, integration, E2E & load tests.
- Penetration testing and vulnerability scanning.
- Draft runbook for SLA breaches and incident response.
- Data retention & GDPR/India PII policy compliance checks.